

COS80022 – Software Quality and Testing

Assignment 2

Student Name: Ajula Peedikaparambil Raju
Student ID: 105286562
Student email: 105286562@student.swin.edu.au
Submission date: 02/11/2025

1.Traceability matrix

Requirements	Test condition	Test case	Script location	Test result
"Add" function	A new Customer's record is successfully added given the valid information	Input data: ['123456', 'Liu', 'Huai', '150', '250', '350'] Expected result: "added"	'test_add' in 'test_assignment2_105286562.py'	Fail
"Add" function	Adding duplicate customer ID is rejected	Input data: ['611822', 'Test', 'User', '100', '100', '100'] Expected result: "not added"	Manual test	Not executed
"Add" function	Invalid customer ID format is rejected	Input data: ['12345', 'Test', 'User', '100', '100', '100'] Expected result: "not added"	Manual test	Not executed
"Add" function	Boundary - Mouse at minimum (0)	Input data: ['888888', 'Test', 'User', '0', '100', '100'] Expected result: 'added'	Manual test	Not executed
"Add" function	Boundary - Mouse at maximum (300)	Input data: ['888889', 'Test', 'User', '300', '100', '100'] Expected result: 'added'	Manual test	Not executed
"Add" function	Boundary - Bag at minimum (0)	Input data: ['888890', 'Test', 'User', '100', '0', '100'] Expected result: 'added'	Manual test	Not executed
"Add" function	Boundary - Lamp at maximum (400)	Input data: ['888891', 'Test', 'User', '100', '100', '400'] Expected result: 'added'	Manual test	Not executed
"Add" function	Invalid - Mouse exceeds	Input data: ['888892', 'Test', 'User', '301',	Manual test	Not executed

Requirements	Test condition	Test case	Script location	Test result
	maximum (301)	'100', '100'] Expected result: 'not added'		
"Add" function	Invalid - Bag negative value (-1)	Input data: ['888893', 'Test', 'User', '100', '-1', '100'] Expected result: 'not added'	Manual test	Not executed
"Add" function	Invalid - Lamp exceeds maximum (401)	Input data: ['888894', 'Test', 'User', '100', '100', '401'] Expected result: 'not added'	Manual test	Not executed
"Add" function	Invalid - Name too short (1 character)	Input data: ['888895', 'A', 'User', '100', '100', '100'] Expected result: 'not added'	Manual test	Not executed
"Add" function	Invalid - Name too long (36 characters)	Input data: ['888896', 'A'*36, 'User', '100', '100', '100'] Expected result: 'not added'	Manual test	Not executed
"Update" function	Existing customer's record is successfully updated	Input data: ['123456', 'Liu', 'Huai', '150', '250', '350'] Expected result: "updated"	'test_update' in 'test_assignment2_105286562.py'	Pass
"Report" function	Report is generated correctly with proper evaluation	Input: Customer ID '123456' Expected: Total 750, "Low Profit"	'test_report' in 'test_assignment2_105286562.py'	Pass
"Report" function	Boundary - Total exactly 499 (High Profit)	Setup: Update customer to total 499 Input: report 123456 Expected result: '499 High Profit'	Manual test	Not executed

Requirements	Test condition	Test case	Script location	Test result
"Report" function	Boundary - Total exactly 500 (Low Profit)	Setup: Update customer to total 500 Input: report 123456 Expected result: '500 Low Profit'	Manual test	Not executed
"Report" function	Boundary - Total exactly 799 (Low Profit)	Setup: Update customer to total 799 Input: report 123456 Expected result: '799 Low Profit'	Manual test	Not executed
"Report" function	Boundary - Total exactly 800 (Loss)	Setup: Update customer to total 800 Input: report 123456 Expected result: '800 Loss'	Manual test	Not executed
"Delete" function	Existing customer record is successfully deleted	Input: Customer ID '123456' Expected result: "deleted"	'test_delete' in 'test_assignment2_105286562.py'	Pass

2.Functional testing

2.1.Test analysis

- **For "Add" function:** *Black-box technique will be used in defining test conditions for this function. Specifically, equivalent partitioning method will be applied.*
 - o Test condition 'add-1': *To demonstrate that a new Customer's record can be successfully added if (1) accurate and comprehensive information is provided; and (2) the Customer's ID does not exist in the "list.csv" file.*
 - o Test condition 'add-2': Check that duplicate customer IDs are rejected (returns "not added").
 - o Test condition 'add-3': Verify customer ID must be exactly 6 digits, reject if not.

Test condition 'add-4': Boundary Value Analysis - Mouse expense at minimum (0) should be accepted.

Test condition 'add-5': Boundary Value Analysis - Mouse expense at maximum (300) should be accepted.

Test condition 'add-6': Boundary Value Analysis - Bag expense at minimum (0) should be accepted.

Test condition 'add-7': Boundary Value Analysis - Lamp expense at maximum (400) should be accepted.

Test condition 'add-8': Boundary Value Analysis - Mouse expense exceeding maximum (301) should be rejected.

Test condition 'add-9': Error Guessing - Negative expense values should be rejected.

Test condition 'add-10': Boundary Value Analysis - Name with 1 character should be rejected.

- **For "Delete" function**
 - o Test condition 'delete-1': Check that existing customer can be deleted successfully.
 - o Test condition 'delete-2': If customer ID doesn't exist, should return "not exist".
- **For "Update" function**
 - o Test condition 'update-1': Verify that existing customer data can be updated with valid new information.
 - o Test condition 'update-2': Check that update fails for non-existent customer ID.
- **For "Report" function**
 - o Test condition 'report-1': Report should calculate total expense correctly (mouse + bag + lamp).
 - o Test condition 'report-2': Check "High Profit" evaluation when total < 500.
 - o Test condition 'report-3': Check "Low Profit" when total is 500-799.
 - o Test condition 'report-4': Check "Loss" when total is 800-1000.

Test condition 'report-5': Boundary Value Analysis - Total expense exactly 499 (upper boundary of High Profit).

Test condition 'report-6': Boundary Value Analysis - Total expense exactly 500 (lower boundary of Low Profit).

Test condition 'report-7': Boundary Value Analysis - Total expense exactly 799 (upper boundary of Low Profit).

Test condition 'report-8': Boundary Value Analysis - Total expense exactly 800 (lower boundary of Loss).

2.2. Test design

- **For "Add" function:** *Black-box technique will be used in designing test cases for this function. Specifically, equivalent partitioning method will be applied.*
 - o Test case for partition 1 (corresponding to test condition 'add-1'): The input data for this test case must contain valid information for all six items and the Customer ID must be new to the existing list, and the record should be successfully added into the list. Thus, the input data is defined as ['123456', 'Liu', 'Huai', '150', '250', '350'], and the expected result (oracle) is "added".

Note: This test failed because ID 123456 already existed in list.csv. In a proper test setup, I should have used a unique ID or cleared the test data first.

- o Test case for 'add-2': Try adding customer with existing ID 611822. Input: ['611822', 'Test', 'User', '100', '100', '100']. Expected: "not added".
- o Test case for 'add-3': Try invalid ID with only 5 digits. Input: ['12345', 'Test', 'User', '100', '100', '100']. Expected: "not added" with error message.

Test case for 'add-4' (Boundary Value Analysis): Input: ['888888', 'Test', 'User', '0', '100', '100']. Expected: 'added'. Tests minimum boundary of mouse expense.

Test case for 'add-5' (Boundary Value Analysis): Input: ['888889', 'Test', 'User', '300', '100', '100']. Expected: 'added'. Tests maximum boundary of mouse expense.

Test case for 'add-6' (Boundary Value Analysis): Input: ['888890', 'Test', 'User', '100', '0', '100']. Expected: 'added'. Tests minimum boundary of bag expense.

Test case for 'add-7' (Boundary Value Analysis): Input: ['888891', 'Test', 'User', '100', '100', '400']. Expected: 'added'. Tests maximum boundary of lamp expense.

Test case for 'add-8' (Boundary Value Analysis): Input: ['888892', 'Test', 'User', '301', '100', '100']. Expected: 'not added'. Tests invalid value above maximum.

Test case for 'add-9' (Error Guessing): Input: ['888893', 'Test', 'User', '100', '-1', '100']. Expected: 'not added'. Tests negative value handling.

Test case for 'add-10' (Boundary Value Analysis): Input: ['888894', 'Test', 'User', '100', '100', '401']. Expected: 'not added'. Tests lamp value above maximum.

Test case for 'add-11' (Boundary Value Analysis): Input: ['888895', 'A', 'User', '100', '100', '100']. Expected: 'not added'. Tests name below minimum length.

- **For "Delete" function**
 - o Test case for 'delete-1': Delete customer ID 123456. Expected: "deleted" and record removed from file. This test passed.
 - o Test case for 'delete-2': Try deleting non-existent ID 000001. Expected: "not exist".
- **For "Update" function**
 - o Test case for 'update-1': Update customer 123456 with ['123456', 'Liu', 'Huai', '150', '250', '350']. Expected: "updated". This test passed.
 - o Test case for 'update-2': Try updating non-existent ID 000002. Expected: "not updated".
- **For "Report" function**
 - o Test case for 'report-1': Report for customer 123456 with expenses 150+250+350=750. Expected: Total 750, evaluation "Low Profit". This test passed.
 - o Test case for 'report-3': Test boundary at 500 (should be "Low Profit") and at 499 (should be "High Profit").
 - o Test case for 'report-4': Test boundary at 800 (should be "Loss").

Test case for 'report-5' (Boundary Value Analysis): Setup: Update to mouse=199, bag=150, lamp=150 (total 499). Input: report 123456. Expected: '499 High Profit'. Tests upper boundary of High Profit.

Test case for 'report-6' (Boundary Value Analysis): Setup: Update to mouse=200, bag=150, lamp=150 (total 500). Input: report 123456. Expected: '500 Low Profit'. Tests lower boundary of Low Profit.

Test case for 'report-7' (Boundary Value Analysis): Setup: Update to mouse=300, bag=299, lamp=200 (total 799). Input: report 123456. Expected: '799 Low Profit'. Tests upper boundary of Low Profit.

Test case for 'report-8' (Boundary Value Analysis): Setup: Update to mouse=300, bag=300, lamp=200 (total 800). Input: report 123456. Expected: '800 Loss'. Tests lower boundary of Loss.

2.3. Test implementation, execution and reporting

- **Test implementation**

All designed test cases have been converted into the pytest script 'test_assignment2_XXXXXX.py'. Their execution order is also scheduled as "test_add", "test_update", "test_report", and finally "test_delete". Such a sequence was deliberately organised (as a test implementation activity) to guarantee the satisfaction of pre-conditions of some test cases. For example, only after adding the Customer information with ID '123456' via the test case "test_add", can we successfully update its information via the test case "test_update".

However, since ID 123456 already existed in the test data, the test_add failed but the other tests still passed because they could use the existing record.

- **Test execution**

```

C:\Users\ajula\OneDrive\Desktop\Sem2\Software Testing\assignmen2pack>python -m pytest test_assignment2_105286562.py -v
===== test session starts =====
platform win32 -- Python 3.11.0, pytest-8.4.2, pluggy-1.6.0 -- C:\Program Files\Python311\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\ajula\OneDrive\Desktop\Sem2\Software Testing\assignmen2pack
plugins: anyio-4.11.0, cov-7.0.0
collected 4 items

test_assignment2_105286562.py::test_add FAILED [ 25%]
test_assignment2_105286562.py::test_update PASSED [ 50%]
test_assignment2_105286562.py::test_report PASSED [ 75%]
test_assignment2_105286562.py::test_delete PASSED [100%]

===== FAILURES =====
test_add

def test_add():
    #The following defines the input data
    filename = "list.csv"
    inputData = ['123456', 'Liu', 'Huai', '150', '250', '350']
    #The following executes the function 'add' with the input data "123456 Liu Huai 150 250 350"
    result = record.addCustomer(filename, inputData)
    #The following verifies if the execution result is consistent with expectation
    > assert result == "added"
E       AssertionError: assert 'not added' == 'added'
E
E       - added
E       + not added

test_assignment2_105286562.py:13: AssertionError
===== short test summary info =====
FAILED test_assignment2_105286562.py::test_add - AssertionError: assert 'not added' == 'added'
===== 1 failed, 3 passed in 0.13s =====
C:\Users\ajula\OneDrive\Desktop\Sem2\Software Testing\assignmen2pack>

```

- **Test results**

Total test cases executed: 4

Passed: 3 (75%)

Failed: 1 (25%)

Detailed Results:

1. test_add - FAILED

Expected result: "added"

Actual result: "not added"

Root cause: Customer ID '123456' already existed in the test data file (list.csv). This is a test environment issue, not a functional defect. The addCustomer function correctly rejected the duplicate ID as per specifications. The function checked that the ID exists (line 27 in record.py) and returned "not added" as designed.

2. test_update - PASSED

The update function successfully modified the customer record for ID '123456'.

All validation checks passed, file was correctly modified, and "updated" was returned.

3. test_report - PASSED

The report function correctly calculated total expense: 150 + 250 + 350 = 750.

Profit evaluation was accurate: 750 falls in range 500-799, correctly evaluated as "Low Profit".

4. test_delete - PASSED

The delete function successfully removed the customer record from list.csv.

The record was properly deleted and "deleted" status was returned.

Fault Analysis:

I didn't find any bugs in the program itself. All four functions work as they should. The test_add failure was my fault for not preparing the test data properly. Next time I should either use a unique customer ID or reset the list.csv file before running tests.

3.Non-functional review

Performance Issues

Issue 1: The checkExist function is slow

Every time the program needs to check if a customer ID exists, it opens the CSV file and reads through all the records line by line. This happens for every add, delete, update, and report operation. If there are lots of customers in the file, this could get really slow.

Why this matters: With 1000 customers, each operation has to read 1000 lines. With 10000 customers, it reads 10000 lines every time. The more customers you have, the slower it gets.

How to fix it:

- Load the CSV into memory once at the start instead of reading it every time
- Use a dictionary to store customer IDs for faster lookup
- For larger systems, use a real database like SQLite instead of CSV

Issue 2: Update and delete rewrite the entire file

When you update or delete one customer, the program reads the entire file, makes the change, and then writes everything back. Even if you're just changing one record, all records get rewritten.

Why this matters: Lots of disk writes are slow and there's a risk of data loss if the program crashes while writing.

How to fix it: Use a database that can update individual records without rewriting everything.

Security Issues

Issue 1: No login system

Anyone who can run the program can add, delete, update, or see all customer data. There's no way to control who does what.

Why this is a problem:

- Unauthorized people could view private customer information
- Someone could delete or change records without permission
- No way to track who made what changes

How to fix it:

- Add username and password login
- Create different user roles (admin can do everything, regular user can only view reports)
- Keep a log file that records who did what and when

Issue 2: Data is not encrypted

Customer names and expense data are stored in plain text. Anyone who can access the list.csv file can read everything.

Why this is a problem: If someone copies the file or accesses the computer, they can see all customer data. This could violate privacy laws.

How to fix it:

- Encrypt the CSV file
- Use a database with built-in encryption
- Set proper file permissions so only authorized users can access it

Issue 3: CSV injection risk

If someone enters a name starting with = or + or -, Excel might treat it as a formula when the CSV is opened. This could potentially run malicious code.

How to fix it: Check input data and add a quote mark before any names starting with =, +, -, or @.

Summary

The program works correctly for its basic functions, but has some issues that would need to be fixed before using it in a real business:

Most important to fix:

- Add authentication and user access control
- Encrypt the customer data

Would be good to improve:

- Make it faster by not reading the file so many times
- Add input validation to prevent CSV injection

For now, the program is OK for small-scale use (maybe a few hundred customers) in a trusted environment where everyone is authorized. But for a production system with sensitive data, these issues should be addressed first.